

CMC Lab 02

Distributed Consistency and Consensus in the Cloud

Lab Documentation Report

Name: Shahd Tamer Abdelmonem (SE3)

1. Overview

This lab explores how distributed systems handle consistency, availability, and consensus under different network conditions. Open-source tools Redis and etcd were used to demonstrate replication and the Raft consensus protocol respectively.

2. Objectives

- Simulate eventual consistency with Redis replicas
- Create a network partition and observe replication behavior
- Experiment with Raft consensus using etcd
- Observe leader election and failover in a Raft cluster

3. Environment Setup

3.1 Prerequisites

- Docker Desktop (installed and running)
- Docker Compose
- Windows PowerShell

3.2 Docker Compose Configuration

The following docker-compose.yml was used to spin up the lab environment:



```

1  services:
2    redis-node1:
3      image: redis:latest
4      container_name: redis-node1
5      ports:
6        - "6379:6379"
7
8    redis-node2:
9      image: redis:latest
10     container_name: redis-node2
11     command: redis-server --replicaof redis-node1 6379
12     ports:
13       - "6380:6379"
14
15   etcd:
16     image: gcr.io/etcd-development/etcd:v3.5.12
17     container_name: etcd
18     command:
19       - /usr/local/bin/etcd
20       - --listen-client-urls=http://0.0.0.0:2379
21       - --advertise-client-urls=http://0.0.0.0:2379
22     ports:
23       - "2379:2379"

```

3.3 Starting the Environment

Navigate to the lab directory and run:

```
cd C:\Users\shahd\OneDrive\Desktop\lab
docker-compose up
```

4. Task 1: Redis Replication and CAP Theorem

4.1 Connecting to Redis Nodes

Two separate PowerShell windows were opened and connected to each Redis node:

1

```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\Users\shahd> docker exec -it redis-node1 redis-cli
127.0.0.1:6379>
```

2

```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\Users\shahd> docker exec -it redis-node2 redis-cli
127.0.0.1:6379>
```

4.2 Writing and Reading Across Nodes

A key was written to redis-node1 and immediately read from redis-node2:

1

```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\Users\shahd> docker exec -it redis-node1 redis-cli
127.0.0.1:6379> SET mykey "hello shahd from node1"
OK
127.0.0.1:6379> |
```

2

```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\Users\shahd> docker exec -it redis-node2 redis-cli
127.0.0.1:6379> GET mykey
"hello shahd from node1"
127.0.0.1:6379> |
```

Output

redis-node2 returned "hello shahd from node1" after a brief replication delay.

What to observe

Eventual consistency — the replica does not update instantly but catches up shortly after the write.

4.3 Simulating a Network Partition

redis-node2 was stopped to simulate a network failure:

```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\Users\shahd> docker stop redis-node2
redis-node2
PS C:\Users\shahd> |
```

A new key was then written to redis-node1 while node2 was down:

```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\Users\shahd> docker exec -it redis-node1 redis-cli
127.0.0.1:6379> SET mykey "hello shahd from node1"
OK
127.0.0.1:6379> SET anotherkey "world"
OK
127.0.0.1:6379>
```

Output

Write to redis-node1 succeeded immediately with OK.

What to observe

Writes to node1 still succeed — Redis prioritizes availability (the "A" in CAP). Node2 is down, so those writes won't replicate until it comes back.

4.4 Recovery After Partition

redis-node2 was restarted and reconnected:

```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\Users\shahd> docker stop redis-node2
redis-node2
PS C:\Users\shahd> docker start redis-node2
redis-node2
PS C:\Users\shahd> |
```

```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\Users\shahd> docker exec -it redis-node2 redis-cli
127.0.0.1:6379> GET mykey
"hello shahd from node1"
127.0.0.1:6379> docker stop redis-node2
(error) ERR unknown command 'docker', with args beginning with: 'stop' 'redis-node2'
127.0.0.1:6379>
What's next:
  Try Docker Debug for seamless, persistent debugging tools in any container or image → docker debug redis-node2
  Learn more at https://docs.docker.com/go/debug-cli/
PS C:\Users\shahd> docker exec -it redis-node2 redis-cli
127.0.0.1:6379>
127.0.0.1:6379> GET anotherkey
"world"
127.0.0.1:6379> |
```

```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\Users\shahd> docker exec -it redis-node1 redis-cli
127.0.0.1:6379> SET mykey "hello shahd from node1"
OK
127.0.0.1:6379> SET anotherkey "world"
OK
127.0.0.1:6379>
```

Output

"world" — node2 successfully synced all missed writes from node1.

What to observe

Node2 now syncs and returns "world". This confirms eventual consistency — the data is consistent eventually after the partition heals.

5. Task 2: Raft Consensus with etcd

5.1 Writing and Reading a Key

A key-value pair was written to etcd and read back:

```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\Users\shahd> docker exec -it etcd etcdctl put foo bar
OK

What's next:
  Try Docker Debug for seamless, persistent debugging tools in any container or image → docker debug etcd
  Learn more at https://docs.docker.com/go/debug-cli/
PS C:\Users\shahd> docker exec -it etcd etcdctl get foo
foo
bar

What's next:
  Try Docker Debug for seamless, persistent debugging tools in any container or image → docker debug etcd
  Learn more at https://docs.docker.com/go/debug-cli/
PS C:\Users\shahd> |
```

Output

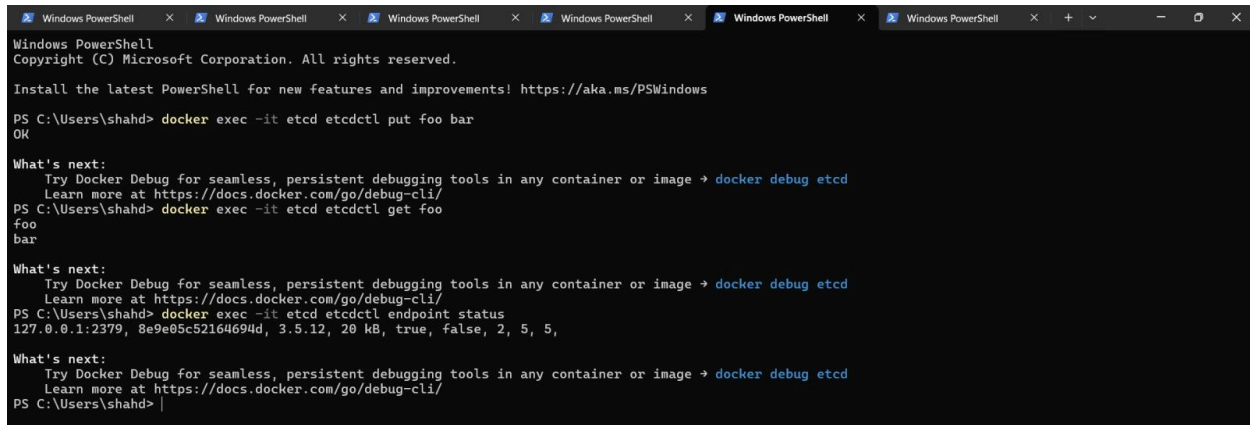
foo bar — the key was confirmed and returned successfully.

What to observe

You get bar back immediately. Unlike Redis replication, etcd uses Raft to guarantee that a write is committed to a majority of nodes before confirming success — this is strong consistency.

5.2 Checking Cluster Leader Status

The endpoint status command was used to inspect the Raft cluster:



```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\Users\shahd> docker exec -it etcd etcdctl put foo bar
OK

What's next:
  Try Docker Debug for seamless, persistent debugging tools in any container or image → docker debug etcd
  Learn more at https://docs.docker.com/go/debug-cli/
PS C:\Users\shahd> docker exec -it etcd etcdctl get foo
foo
bar

What's next:
  Try Docker Debug for seamless, persistent debugging tools in any container or image → docker debug etcd
  Learn more at https://docs.docker.com/go/debug-cli/
PS C:\Users\shahd> docker exec -it etcd etcdctl endpoint status
127.0.0.1:2379, 8e9e05c52164694d, 3.5.12, 20 kB, true, false, 2, 5, 5,

What's next:
  Try Docker Debug for seamless, persistent debugging tools in any container or image → docker debug etcd
  Learn more at https://docs.docker.com/go/debug-cli/
PS C:\Users\shahd> |
```

Output

Displays the endpoint URL, node ID, current Raft term, Raft index, and whether the node is the leader.

What to observe

A new leader is elected automatically within a few seconds. This is Raft's leader election — it ensures the cluster keeps working as long as a majority (quorum) of nodes are alive.

5.3 Simulating Leader Failure

The etcd container was stopped to simulate a leader crash:

```

Mode                LastWriteTime         Length Name
-----
-a---l            5/2/2026   7:51 PM           592 docker-compose.yml

PS C:\Users\shahd\OneDrive\Desktop\lab> docker-compose up
time="2026-05-02T19:52:07+03:00" level=warning msg="C:\\Users\\shahd\\OneDrive\\Desktop\\lab\\docker-compose.yml: the attribute 'version' is obsolete, it will be ignored, please remove it to avoid potential confusion"
unable to get image 'bitnami/etcd:latest': failed to connect to the docker API at npipe:////./pipe/dockerDesktopLinuxEngine; check if the path is correct and if the daemon is running: open //./pipe/dockerDesktopLinuxEngine: The system cannot find the file specified.
PS C:\Users\shahd\OneDrive\Desktop\lab> docker-compose up
time="2026-05-02T19:54:18+03:00" level=warning msg="C:\\Users\\shahd\\OneDrive\\Desktop\\lab\\docker-compose.yml: the attribute 'version' is obsolete, it will be ignored, please remove it to avoid potential confusion"
[+] up 2/2
! Image redis:7 Interrupted 15.9s
! Image bitnami/etcd:latest Interrupted 15.9s
Error response from daemon: failed to resolve reference "docker.io/library/redis:7": failed to do request: Head "https://registry-1.docker.io/v2/library/redis/manifests/7": net/http: TLS handshake timeout
PS C:\Users\shahd\OneDrive\Desktop\lab> docker-compose up
time="2026-05-02T19:58:22+03:00" level=warning msg="C:\\Users\\shahd\\OneDrive\\Desktop\\lab\\docker-compose.yml: the attribute 'version' is obsolete, it will be ignored, please remove it to avoid potential confusion"
[+] up 2/2
! Image redis:7 Interrupted 2.7s
X Image bitnami/etcd:latest Error failed to resolve reference "docker.io/bitnami/etcd:latest": docker.io/... 2.7s
Error response from daemon: failed to resolve reference "docker.io/bitnami/etcd:latest": docker.io/bitnami/etcd:latest: not found
PS C:\Users\shahd\OneDrive\Desktop\lab> docker-compose up
[+] up 19/19
✓ Image gcr.io/etcd-development/etcd:v3.5.12 Pulled 28.9s
✓ Network lab_default Created 0.1s
✓ Container redis-node1 Created 0.6s
✓ Container etcd Created 0.6s
✓ Container redis-node2 Created 0.6s
Attaching to etcd, redis-node1, redis-node2
etcd | {"level":"warn","ts":"2026-05-02T17:10:37.274093Z","caller":"embed/config.go:679","msg":"Running http and grpc server on single port. This is not recommended for production."}
etcd | {"level":"info","ts":"2026-05-02T17:10:37.274219Z","caller":"etcdmain/etcd.go:73","msg":"Running: ", "args":["/usr/local/bin/etcd", "--listen-client-urls=http://0.0.0.0:2379", "--advertise-client-urls=http://0.0.0.0:2379"]}
etcd | {"level":"warn","ts":"2026-05-02T17:10:37.274244Z","caller":"etcdmain/etcd.go:105","msg":"'data-dir' was empty; using default", "data-dir":"default.etcd"}

```

Output

After restarting, etcd rejoined the cluster. The Raft term incremented, indicating a new election took place.

What to observe

Raft guarantees automatic leader re-election when the current leader becomes unavailable, ensuring the cluster remains operational as long as a majority of nodes are alive.

6. Summary of Findings

Concept	Tool Used	Observed Behavior
Eventual Consistency	Redis	Node2 briefly returned nil before syncing with node1
Availability (CAP)	Redis	Node1 continued accepting writes while node2 was down
Data Recovery	Redis	Node2 re-synced all missed writes upon restart
Strong Consistency	etcd	Write confirmed only after Raft majority agreement
Leader Election	etcd	New leader elected automatically after container stop

7. Conclusion

This lab demonstrated the practical trade-offs described by the CAP theorem through hands-on experimentation with Redis and etcd.

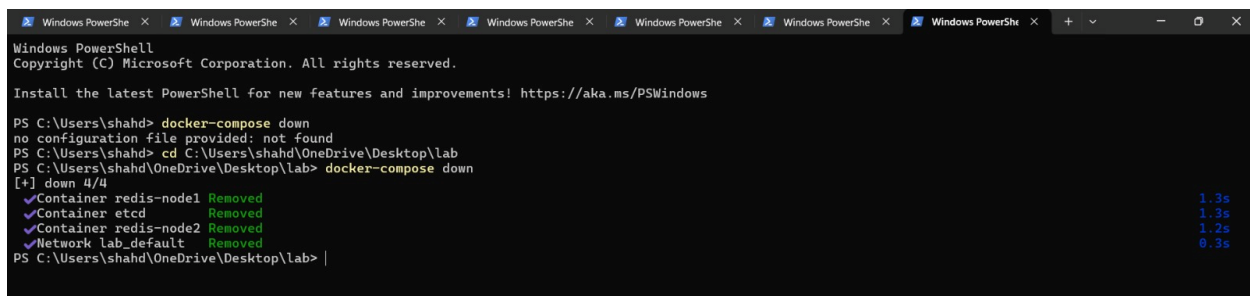
Redis, configured in a primary-replica setup, prioritizes Availability — it continues to function and accept writes even when replicas are unreachable, eventually synchronizing data once connectivity is restored. This makes Redis well-suited for use cases where uptime is critical and brief data inconsistency is tolerable.

etcd, using the Raft consensus protocol, prioritizes Consistency — every write is confirmed only after agreement from a majority of cluster nodes. This guarantees that all reads reflect the most recent committed state, making etcd ideal for critical coordination tasks such as service discovery, distributed locking, and configuration management.

Understanding these trade-offs is essential when designing distributed systems, as the choice of consistency model directly impacts the system's behavior during network failures and node outages.

8. Cleanup

After completing all tasks, the lab environment was shut down:



```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\Users\shahd> docker-compose down
no configuration file provided: not found
PS C:\Users\shahd> cd C:\Users\shahd\OneDrive\Desktop\lab
PS C:\Users\shahd\OneDrive\Desktop\lab> docker-compose down
[+] down 4/4
✓Container redis-node1 Removed 1.3s
✓Container etcd Removed 1.3s
✓Container redis-node2 Removed 1.2s
✓Network lab_default Removed 0.3s
PS C:\Users\shahd\OneDrive\Desktop\lab> |
```